Ritucharya Recommendation Engine: Complete Flowcharts & Diagrams

Table of Contents

---

1. How `.find()` Works

Iteration Process

```
ARRAY TO SEARCH:
┌─────────────────────────────────────────────────────────┐
│ prakritiData = [                                        │
│   { season: 'Hemanta', recommendations: {...} },        │
│   { season: 'Shishira', recommendations: {...} },       │
│   { season: 'Vasanta', recommendations: {...} },        │
│   { season: 'Grishma', recommendations: {...} },  ← TARGET│
│   { season: 'Varsha', recommendations: {...} },         │
│   { season: 'Sharad', recommendations: {...} }          │
│ ]                                                       │
└─────────────────────────────────────────────────────────┘

SEARCH FOR: season = 'Grishma'

ITERATION:
┌─────┬──────────────┬──────────────────────┬──────────┬──────────────┐
│ i   │ Element      │ Check                │ Result   │ Action       │
├─────┼──────────────┼──────────────────────┼──────────┼──────────────┤
│ 0   │ Hemanta      │ Hemanta === Grishma? │ ❌ NO    │ Continue     │
│ 1   │ Shishira     │ Shishira === Grishma?│ ❌ NO    │ Continue     │
│ 2   │ Vasanta      │ Vasanta === Grishma? │ ❌ NO    │ Continue     │
│ 3   │ Grishma      │ Grishma === Grishma? │ ✅ YES   │ RETURN ⟵     │
│ 4   │ (Never reach)│                      │ -        │ -            │
│ 5   │ (Never reach)│                      │ -        │ -            │
└─────┴──────────────┴──────────────────────┴──────────┴──────────────┘

RETURNS:
{
```

```
  season: 'Grishma',
  recommendations: {
    diet: ["cool liquids", "coconut water"],
    lifestyle: ["swimming", "cool showers"],
    avoid: ["spicy foods", "direct sun"]
  }
}
```

Without `.find()` (Equivalent Code)

```
// What .find() does internally:

let seasonalRec = null;
for (let i = 0; i < prakritiData.length; i++) {
  if (prakritiData[i].season === 'Grishma') {
    seasonalRec = prakritiData[i];
    break;  // Stop immediately when found
  }
}
return seasonalRec;
```

Pseudocode

```
FUNCTION find(array, testCondition):
  FOR EACH element IN array:
    IF testCondition(element) is TRUE:
      RETURN element  ← Exit immediately
    END IF
  END FOR
  RETURN null  ← If nothing found
```

---

2. JSON Files to Memory

File System Structure

```
backend/
│
├── recommendationEngine.js  (Main logic)
│
├── vata.json               ← 6 seasonal entries
│   [
│     {season: 'Hemanta', recommendations: {...}},
│     {season: 'Shishira', recommendations: {...}},
│     ...
│   ]
```

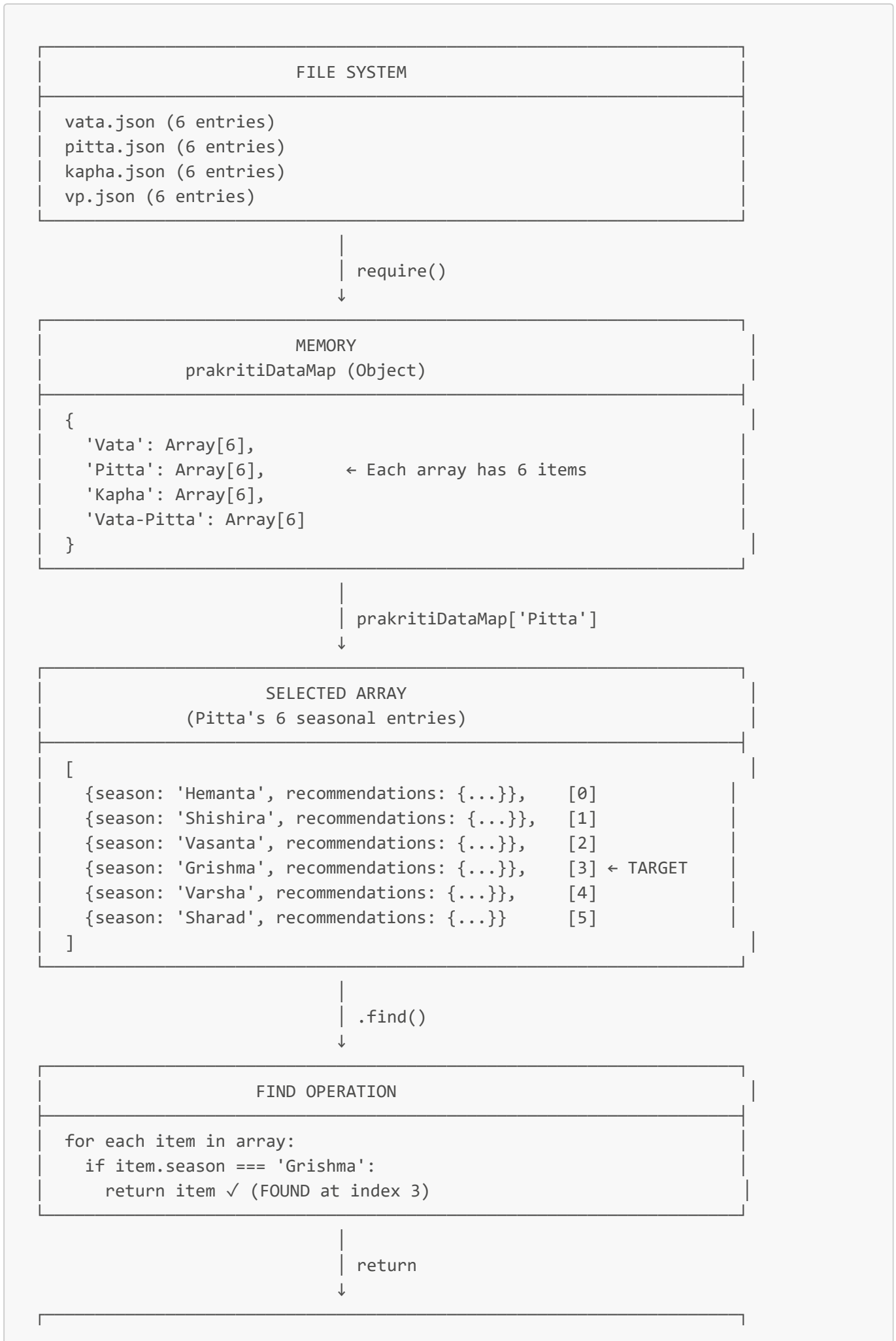
```
│
├── pitta.json              ← 6 seasonal entries
│   [
│     {season: 'Hemanta', recommendations: {...}},
│     ...
│   ]
│
├── kapha.json              ← 6 seasonal entries
│   [...]
│
└── vp.json                 ← 6 seasonal entries (Vata-Pitta)
    [...]
```

Loading Process

```
STEP 1: Load Individual Files
──────────────────────────────
const vataData = require('./vata.json');     → Array[6]
const pittaData = require('./pitta.json');   → Array[6]
const kaphaData = require('./kapha.json');   → Array[6]
const vpData = require('./vp.json');         → Array[6]


STEP 2: Create Map (Dictionary)
────────────────────────────────
const prakritiDataMap = {
  'Vata':        vataData,      ← Points to vata.json array
  'Pitta':       pittaData,     ← Points to pitta.json array
  'Kapha':       kaphaData,     ← Points to kapha.json array
  'Vata-Pitta':  vpData         ← Points to vp.json array
};


STEP 3: Access Specific Array
──────────────────────────────
const prakritiData = prakritiDataMap['Pitta'];
↓
Now prakritiData = Array[6] from pitta.json


STEP 4: Search Within That Array
─────────────────────────────────
seasonalRec = prakritiData.find(rec => rec.season === 'Grishma')
↓
Returns matching object from that specific array
```

---

3. Complete Flow Diagram


FILE SYSTEM
vata.json (6 entries)
pitta.json (6 entries)
kapha.json (6 entries)
vp.json (6 entries)
require()
MEMORY
prakritiDataMap (Object)
{
'Vata': Array[6],
'Pitta': Array[6], ← Each array has 6 items
'Kapha': Array[6],
'Vata-Pitta': Array[6]
}
prakritiDataMap['Pitta']
SELECTED ARRAY
(Pitta's 6 seasonal entries)
[
{season: 'Hemanta', recommendations: {...}}, [0]
{season: 'Shishira', recommendations: {...}}, [1]
{season: 'Vasanta', recommendations: {...}}, [2]
{season: 'Grishma', recommendations: {...}}, [3] ← TARGET
{season: 'Varsha', recommendations: {...}}, [4]
{season: 'Sharad', recommendations: {...}} [5]
]
.find()
FIND OPERATION
for each item in array:
if item.season === 'Grishma':
return item ✓ (FOUND at index 3)
return

```
┌──────────────────────────────────────────────────────────────────┐
│                          RESULT                                  │
├──────────────────────────────────────────────────────────────────┤
│  {                                                               │
│    season: 'Grishma',                                            │
│    recommendations: {                                            │
│      diet: ["cool liquids", "coconut water", ...],               │
│      lifestyle: ["swimming", "cool showers", ...],               │
│      avoid: ["spicy foods", "direct sun", ...]                   │
│    }                                                             │
│  }                                                               │
└──────────────────────────────────────────────────────────────────┘
                              │
                              │ Backend sends to Frontend
                              ↓
┌──────────────────────────────────────────────────────────────────┐
│                     FRONTEND DISPLAYS                            │
├──────────────────────────────────────────────────────────────────┤
│  ✅ Diet recommendations                                         │
│  ✅ Lifestyle recommendations                                    │
│  ✅ Foods to avoid                                               │
└──────────────────────────────────────────────────────────────────┘
```

---

4. Memory Layout

```
prakritiDataMap (Object in Memory)
│
├──────────────────────────────────────────────────────────────────┐
│ 'Vata' → vataData (Array)
│          │
│          ├─ [0] { season: 'Hemanta', recommendations: {...} }
│          ├─ [1] { season: 'Shishira', recommendations: {...} }
│          ├─ [2] { season: 'Vasanta', recommendations: {...} }
│          ├─ [3] { season: 'Grishma', recommendations: {...} }
│          ├─ [4] { season: 'Varsha', recommendations: {...} }
│          └─ [5] { season: 'Sharad', recommendations: {...} }
│
├──────────────────────────────────────────────────────────────────┐
│ 'Pitta' → pittaData (Array)
│           │
│           ├─ [0] { season: 'Hemanta', recommendations: {...} }
│           ├─ [1] { season: 'Shishira', recommendations: {...} }
│           ├─ [2] { season: 'Vasanta', recommendations: {...} }
│           ├─ [3] { season: 'Grishma', recommendations: {...} }
│           ├─ [4] { season: 'Varsha', recommendations: {...} }
│           └─ [5] { season: 'Sharad', recommendations: {...} }
│
├──────────────────────────────────────────────────────────────────┐
│ 'Kapha' → kaphaData (Array)
│           │
│           ├─ [0-5] seasonal entries...
```

```
│
├──────────────────────────────────────────────────────────────────────┐
│ 'Vata-Pitta' → vpData (Array)
│                 │
│                 └─ [0-5] seasonal entries...
│
└──────────────────────────────────────────────────────────────────────┘

Total Structure:
- 1 Object (prakritiDataMap)
- 4 Arrays (one per prakriti type)
- 24 Objects total (4 × 6 seasonal entries)
```

---

5. Real Example with Numbers

Scenario: May 25, 2026, User is Pitta

```
STEP 1: Get Current Season
──────────────────────────

getCurrentSeason()
│
├─ today = May 25, 2026
├─ month = 5 (May)
├─ Check: IF month ∈ {5, 6} → YES
└─ Return: 'Grishma'


STEP 2: Get User's Prakriti Data
────────────────────────────────

prakritiType = 'Pitta'
prakritiData = prakritiDataMap['Pitta']
│
└─ Now prakritiData points to pittaData (Array[6])


STEP 3: Search for Matching Season
──────────────────────────────────

season = 'Grishma'

prakritiData.find(rec => rec.season === 'Grishma')
│
├─ Index 0: rec.season = 'Hemanta' — 'Hemanta' === 'Grishma'? NO
├─ Index 1: rec.season = 'Shishira' — 'Shishira' === 'Grishma'? NO
├─ Index 2: rec.season = 'Vasanta' — 'Vasanta' === 'Grishma'? NO
├─ Index 3: rec.season = 'Grishma' — 'Grishma' === 'Grishma'? YES ✓
│
└─ RETURN this object


STEP 4: Return Recommendations
```

```
──────────────────────────────────
{
  "season": "Grishma",
  "season_role": "HIGHLY AGGRAVATING",
  "weather_characteristics": ["heat", "high sun"],
  "recommendations": {
    "diet": [
      "cool liquids",
      "coconut water",
      "sweet & bitter tastes",
      "cucumber",
      "watermelon"
    ],
    "lifestyle": [
      "swimming",
      "cool showers",
      "evening walks",
      "avoid direct sun"
    ],
    "avoid": [
      "spicy foods",
      "direct sun exposure",
      "overwork",
      "hot environments"
    ]
  },
  "reasoning": {
    "principle": "Samanya-Vishesha Siddhanta",
    "dosha_effect": "Cooling qualities balance naturally hot Pitta"
  }
}


STEP 5: Send to Frontend
──────────────────────────
Frontend receives this object
│
├─ Displays Season: Grishma (Summer)
├─ Displays Prakriti: Pitta (Hot, Intense)
├─ Displays Diet: Cool liquids, coconut water, etc.
├─ Displays Lifestyle: Swimming, cool showers, etc.
└─ Displays Avoid: Spicy foods, direct sun, etc.
```

---

6. Find Operation Detailed

What `.find()` Actually Does

```
const seasonalRec = prakritiData.find(rec => rec.season === season);
```

Breaking it down:

```
prakritiData.find()
├─ Method: find
├─ Purpose: Iterate through array and return first match
├─ Parameter: rec => rec.season === season
│           (Arrow function that tests each element)
│
├─ For Each Element:
│  ├─ rec = current element
│  ├─ Test: rec.season === season
│  ├─ If TRUE: Return rec immediately (stop loop)
│  └─ If FALSE: Continue to next element
│
└─ If No Match Found: Return undefined
```

Step-by-Step Execution

```
GIVEN:
prakritiData = [
  {season: 'Hemanta'},
  {season: 'Shishira'},
  {season: 'Vasanta'},
  {season: 'Grishma'},    ← We're looking for this
  {season: 'Varsha'},
  {season: 'Sharad'}
]

season = 'Grishma'

EXECUTION:

Step 1:
  rec = prakritiData[0]
  rec.season = 'Hemanta'
  Test: 'Hemanta' === 'Grishma'? → FALSE
  Action: Continue to next

Step 2:
  rec = prakritiData[1]
  rec.season = 'Shishira'
  Test: 'Shishira' === 'Grishma'? → FALSE
  Action: Continue to next

Step 3:
  rec = prakritiData[2]
  rec.season = 'Vasanta'
  Test: 'Vasanta' === 'Grishma'? → FALSE
  Action: Continue to next
```

```
Step 4:
  rec = prakritiData[3]
  rec.season = 'Grishma'
  Test: 'Grishma' === 'Grishma'? → TRUE ✓
  Action: RETURN {season: 'Grishma', ...}

  ⚠️ LOOP STOPS HERE - No further iterations


RESULT: {season: 'Grishma', recommendations: {...}}
```

---

7. Season Mapping

Month to Season Function

```
January  (1)   → Shishira (Cold)
February (2)   → Shishira (Cold)
March    (3)   → Vasanta  (Spring)
April    (4)   → Vasanta  (Spring)
May      (5)   → Grishma  (Summer) ← Today
June     (6)   → Grishma  (Summer)
July     (7)   → Varsha   (Monsoon)
August   (8)   → Varsha   (Monsoon)
September(9)   → Varsha   (Monsoon)
October  (10)  → Sharad   (Autumn)
November (11)  → Hemanta  (Cool/Dewy)
December (12)  → Hemanta  (Cool/Dewy)


VISUAL TIMELINE:
┌──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┐
│J │F │M │A │M │J │J │A │S │O │N │D│
├──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┤
│Sh│Sh│Va│Va│Gr│Gr│Va│Va│Va│Sh│He│He│
└──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┘
  Shi  Vas  Gri  Var  Sha  Hem
  (Cold)(Spri)(Sum)(Mon)(Aut)(Cool)
                ↑
              Today
```

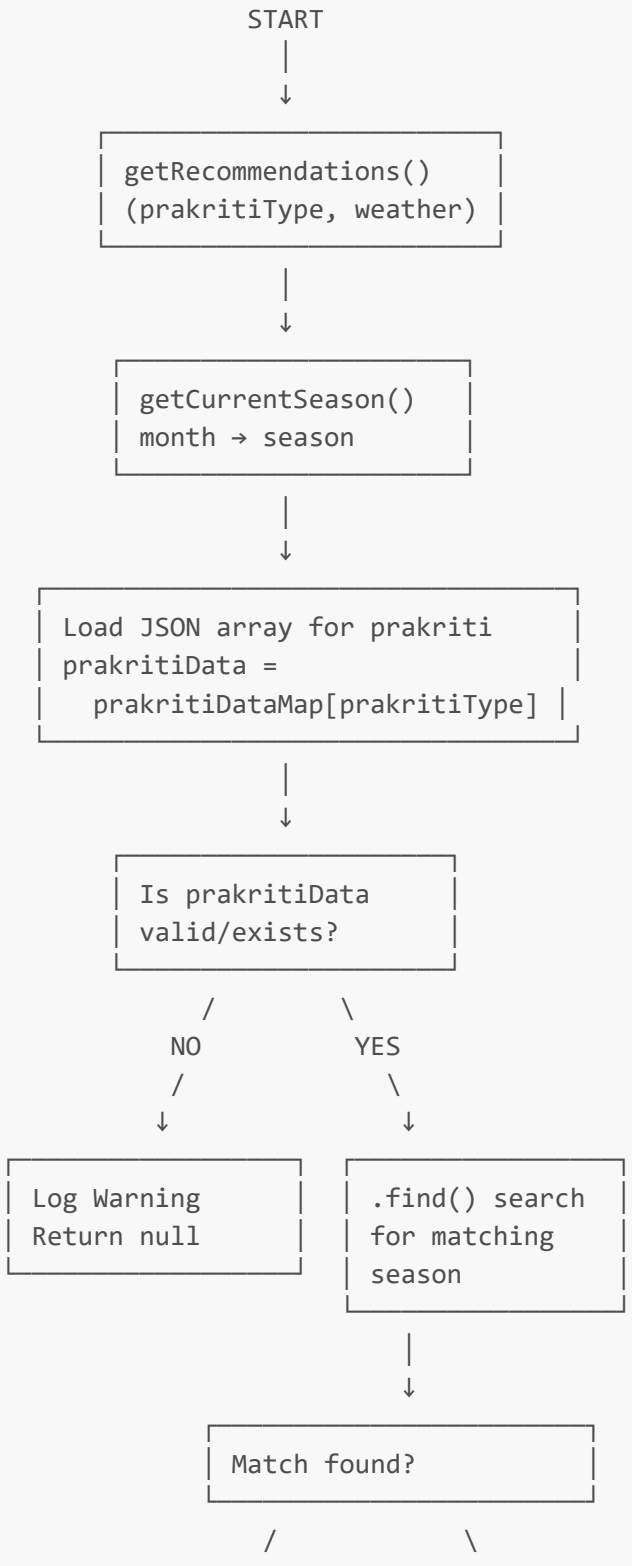
JavaScript Implementation

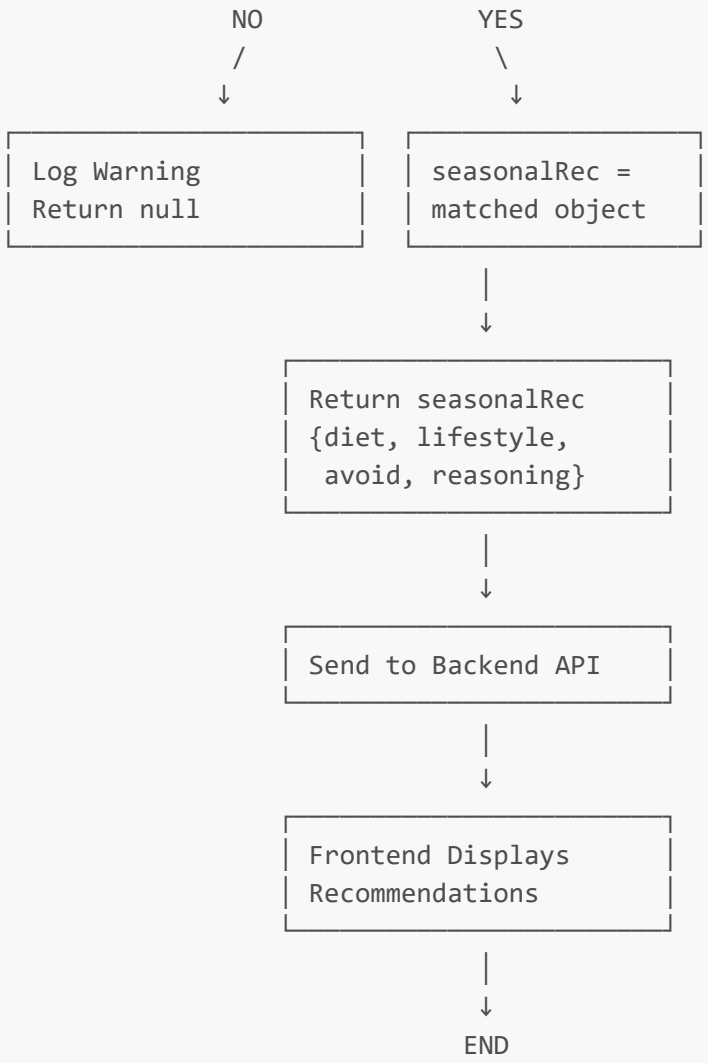
```
function getCurrentSeason() {
  const month = new Date().getMonth() + 1; // 1-12

  if (month >= 11 || month === 12) return 'Hemanta';
  else if (month === 1 || month === 2) return 'Shishira';
  else if (month === 3 || month === 4) return 'Vasanta';
```

```
  else if (month === 5 || month === 6) return 'Grishma';
  else if (month >= 7 && month <= 9) return 'Varsha';
  else if (month === 10) return 'Sharad';

  return 'Varsha'; // Default fallback
}
```

---

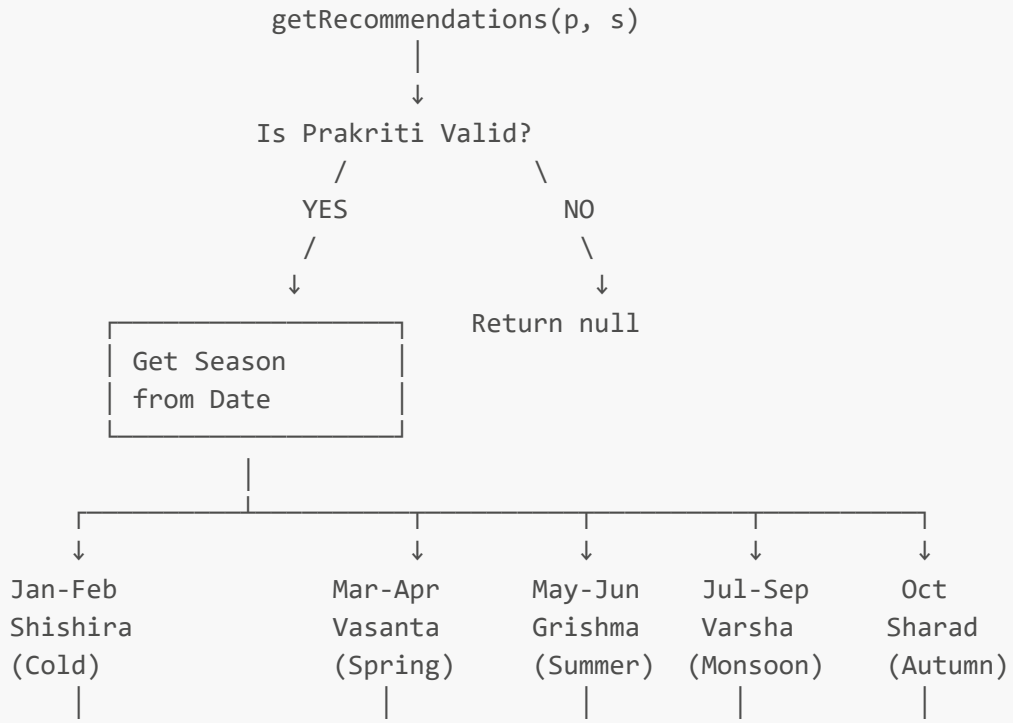
8. Algorithm Flowchart

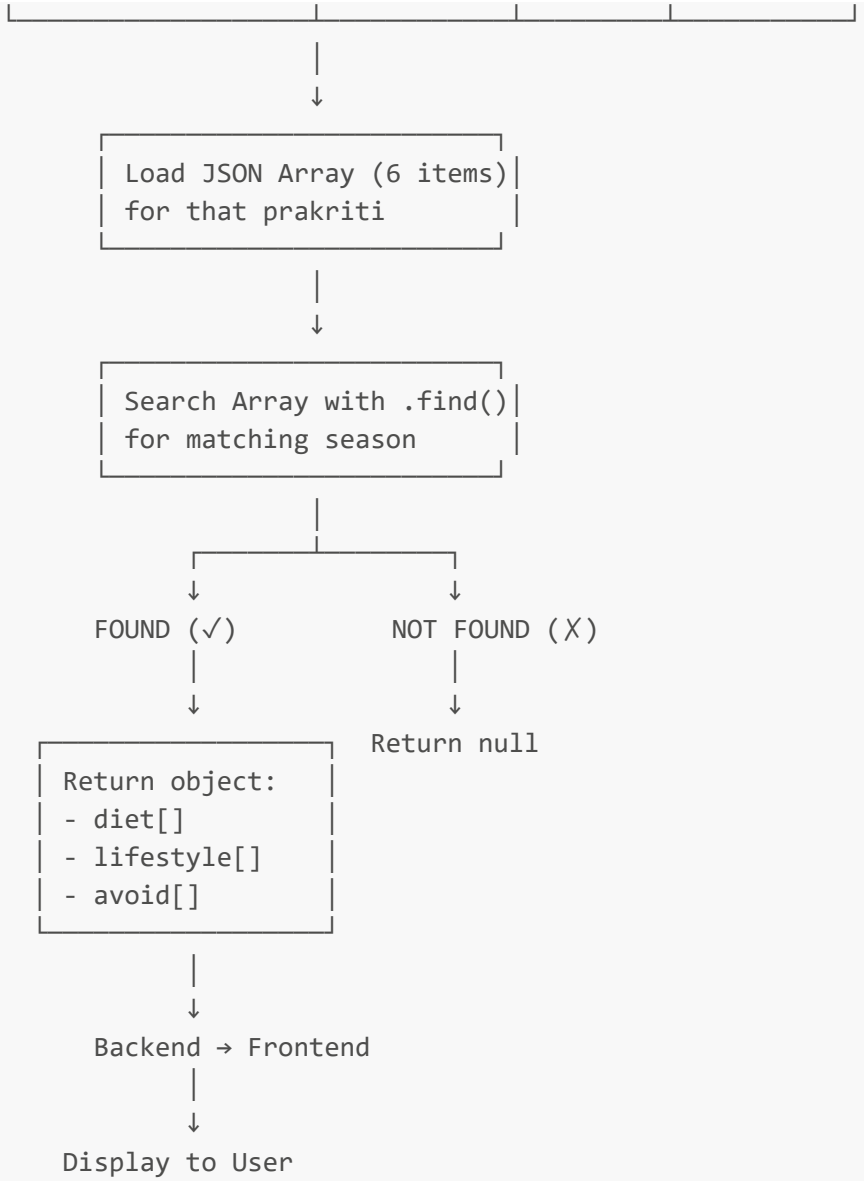




---

9. Decision Tree





Summary Table

Step	Action	Input	Output
1	`getCurrentSeason()`	Date	Season (string)
2	Load from map	prakritiType	Array[6]
3	`.find()` search	season, array	Object or null
4	Return	Object	Recommendations
5	Send to API	Recommendations	JSON response
6	Display	JSON	User sees recommendations

Key Files Reference

File	Purpose	Contents
recommendationEngine.js	Main logic	Functions + map
vata.json	Vata recommendations	6 seasonal entries
pitta.json	Pitta recommendations	6 seasonal entries
kapha.json	Kapha recommendations	6 seasonal entries
vp.json	Vata-Pitta recommendations	6 seasonal entries

Code Snippet Reference

```
// The complete function:
function getRecommendations(prakritiType, weather) {
  try {
    const season = getCurrentSeason();  // Step 1
    const prakritiData = prakritiDataMap[prakritiType];  // Step 2

    if (!prakritiData) {
      console.warn(`No data found for prakriti type: ${prakritiType}`);
      return null;
    }

    const seasonalRec = prakritiData.find(  // Step 3 - .find()
      rec => rec.season === season
    );

    if (!seasonalRec) {
      console.warn(`No recommendations found for ${prakritiType} in ${season}`);
      return null;
    }

    return seasonalRec;  // Step 4
  } catch (error) {
    console.error('Error in getRecommendations:', error);
    return null;
  }
}
```

Document Created: May 25, 2026
Flowchart Type: Complete system architecture
Use Case: Understanding recommendation engine flow